NAME-Siddhi Dipak Jagtap
PRN-123B1B147

//INFIX TO PREFIX CODE

```
#include <iostream>
#include <cstring>
#include <stack>
using namespace std;

// Function to return precedence of operators

int prec(char c) {
   if (c == '^')
      return 3;
   else if (c == '/' || c == '*')
      return 2;
   else if (c == '+' || c == '-')
      return 1;
   else
      return -1;
}

// Function to reverse a string

void reverseString(char* str) {
   int n = strlen(str);
   for (int i = 0; i < n / 2; i++) {
      swap(str[i], str[n - i - 1]);
   }
}

// Function to convert infix expression to postfix expression

void infixToPostfix(char s[], char* postfix) {
   int postIndex = 0;
   int len = strlen(s);
   char stack[1000];
   int stacktop = -1;

   for (int i = 0; i < len; i++) {
      char c = s[i];

      // If the scanned character is an operand, add it to the output string
      if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9')) {
         postfix[postIndex++] = c;
      }
      // If the scanned character is an '(', push it to the stack.
      else if (c == '(') {
         stack[++stacktop] = c;
      }
```

```
        // If the scanned character is an ‘)’, pop and add to the output string from the stack until an ‘(‘ is
encountered.
        else if (c == ')') {
            while (stacktop >= 0 && stack[stacktop] != '(') {
                postfix[postIndex++] = stack[stacktop--];
            }
            stacktop--; // Pop '('
        }
        // If an operator is scanned

        else {
            while (stacktop >= 0 && (prec(c) <= prec(stack[stacktop]))) {
                postfix[postIndex++] = stack[stacktop--];
            }
            stack[++stacktop] = c;
        }
    }

    // Pop all the remaining elements from the stack

    while (stacktop >= 0) {
        postfix[postIndex++] = stack[stacktop--];
    }

    postfix[postIndex] = '\0';
}

// Function to convert infix to prefix expression

void infixToPrefix(char* infix) {
    // Step 1: Reverse the infix expression
    reverseString(infix);

    // Replace '(' with ')' and vice versa
    for (int i = 0; i < strlen(infix); i++) {
        if (infix[i] == '(') {
            infix[i] = ')';
        } else if (infix[i] == ')') {
            infix[i] = '(';
        }
    }

    //  Convert the modified infix expression to postfix

    char postfix[1000];
    infixToPostfix(infix, postfix);

    //  Reverse the postfix expression to get the prefix expression

    reverseString(postfix);
```

```
    cout << "Prefix Expression: " << postfix << endl;
}

// Driver code
int main() {
    char exp[] = "(a+b)*(c^d-e)^(f+g*h)-i";

    // Function call to convert infix to prefix
    infixToPrefix(exp);

    return 0;
}
```